

Handwritten Digit Recognition with LeNet5 Model in PyTorch

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch**  4

[Share](#)[Share](#)[Post](#)

A popular demonstration of the capability of deep learning techniques is object recognition in image data. The “hello world” of object recognition for machine learning and deep learning is the MNIST dataset for handwritten digit recognition. In this post, you will discover how to develop a deep learning model to achieve near state-of-the-art performance on the MNIST handwritten digit recognition task in PyTorch. After completing this chapter, you will know:

- How to load the MNIST dataset using torchvision
- How to develop and evaluate a baseline neural network model for the MNIST problem
- How to implement and evaluate a simple Convolutional Neural Network for MNIST
- How to implement a state-of-the-art deep learning model for MNIST

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Handwritten Digit Recognition with LeNet5 Model in PyTorch
Photo by Johnny Wong. Some rights reserved.

Overview

This post is divided into five parts; they are:

- The MNIST Handwritten Digit Recognition Problem
- Loading the MNIST Dataset in PyTorch
- Baseline Model with Multilayer Perceptrons

- Simple Convolutional Neural Network for MNIST
- LeNet5 for MNIST

The MNIST Handwritten Digit Recognition Problem

The MNIST problem is a classic problem that can demonstrate the power of convolutional neural networks. The MNIST dataset was developed by Yann LeCun, Corinna Cortes, and Christopher Burges for evaluating machine learning models on the handwritten digit classification problem. The dataset was constructed from a number of scanned document datasets available from the National Institute of Standards and Technology (NIST). This is where the name for the dataset comes from, the Modified NIST or MNIST dataset.

Images of digits were taken from a variety of scanned documents, normalized in size, and centered. This makes it an excellent dataset for evaluating models, allowing the developer to focus on machine learning with minimal data cleaning or preparation required. Each image is a 28×28-pixel square (784 pixels total) in grayscale. A standard split of the dataset is used to evaluate and compare models, where 60,000 images are used to train a model, and a separate set of 10,000 images are used to test it.

To goal of this problem is to identify the digits on the image. There are ten digits (0 to 9) or ten classes to predict. The state-of-the-art prediction accuracy is at 99.8% level, achieved with large convolutional neural networks.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

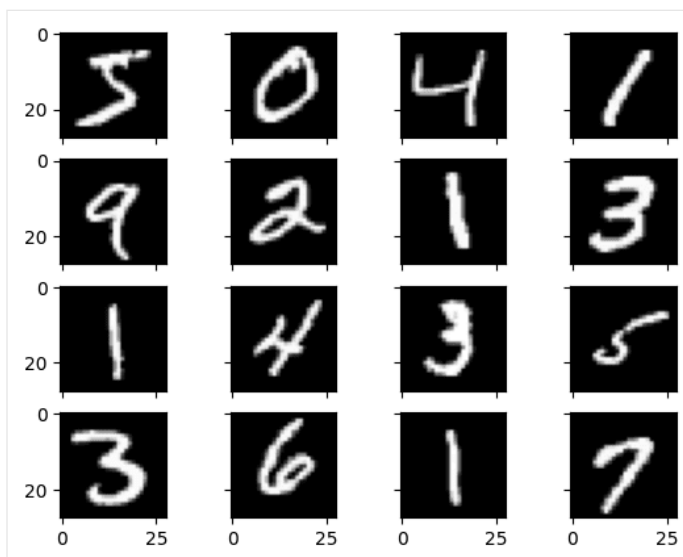
Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Loading the MNIST Dataset in PyTorch

The torchvision library is a sister project of PyTorch that provide specialized functions for computer vision tasks. There is a function in torchvision that can download the MNIST dataset for use with PyTorch. The dataset is downloaded the first time this function is called and stored locally, so you don't need to download again in the future. Below is a little script to download and visualize the first 16 images in the training subset of the MNIST dataset.

```
1 import matplotlib.pyplot as plt
2 import torchvision
3
4 train = torchvision.datasets.MNIST('./data', train=True, download=True)
5
6 fig, ax = plt.subplots(4, 4, sharex=True, sharey=True)
7 for i in range(4):
8     for j in range(4):
9         ax[i][j].imshow(train.data[4*i+j], cmap="gray")
10 plt.show()
```



Baseline Model with Multilayer Perceptrons

Do you really need a complex model like a convolutional neural network to get the best results with MNIST? You can get good results using a very simple neural network model with a single hidden layer. In this section, you will create a simple multilayer perceptron model that achieves accuracy of 99.81%. You will use this as a baseline for comparison to more complex convolutional neural network models. First, let's check what the data looks like:

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision
5
6 # Load MNIST data
7 train = torchvision.datasets.MNIST('data', train=True, download=True)
8 test = torchvision.datasets.MNIST('data', train=False, download=True)
9 print(train.data.shape, train.targets.shape)
10 print(test.data.shape, test.targets.shape)
```

You should see:

```
1 torch.Size([60000, 28, 28]) torch.Size([60000])
2 torch.Size([10000, 28, 28]) torch.Size([10000])
```

The training dataset is structured as a 3-dimensional array of instance, image height, and image width. For a multilayer perceptron model, you must reduce the images down into a vector of pixels. In this case, the 28x28-sized images will be 784 pixel input vectors. You can do this transform easily using the `reshape()` function.

The pixel values are grayscale between 0 and 255. It is almost always a good idea to perform some scaling of input values when using neural network models. Because the scale is well known and well behaved, you can very quickly normalize the pixel values to the range 0 and 1 by dividing each value by the maximum of 255.

In the following, you transform the dataset, convert to floating point, and normalize them by scaling floating point values and you can normalize them easily in the next step.

```
1 # each sample becomes a vector of values 0-1
2 X_train = train.data.reshape(-1, 784).float() / 255.0
3 y_train = train.targets
4 X_test = test.data.reshape(-1, 784).float() / 255.0
5 y_test = test.targets
```

The output targets `y_train` and `y_test` are labels in the form of integers from 0 to 9. This is a multiclass classification problem. You can convert these labels into one-hot encoding or keep them as integer labels like this case. You are going to use the cross entropy function to evaluate the model performance and the PyTorch implementation of cross entropy function can be applied on one-hot encoded targets or integer labeled targets.

You are now ready to create your simple neural network model. You will define your model in a PyTorch `Module` class.

```
1 class Baseline(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.layer1 = nn.Linear(784, 784)
5         self.act1 = nn.ReLU()
6         self.layer2 = nn.Linear(784, 10)
7
8     def forward(self, x):
9         x = self.act1(self.layer1(x))
10        x = self.layer2(x)
11        return x
```

The model is a simple neural network with one hidden layer with the same number of neurons as there are inputs (784). A rectifier activation function is used for the neurons in the hidden layer. The output of this model are **logits**, meaning they are real numbers which can be transformed into probability-like values using a softmax function. You do not apply the softmax function explicitly because the cross entropy function will do that for you.

You will use the stochastic gradient descent algorithm (with learning rate set to 0.01) to optimize this model. The training loop is as follows:

```
1 model = Baseline()
2
3 optimizer = optim.SGD(model.parameters(), lr=0.01)
4 loss_fn = nn.CrossEntropyLoss()
5 loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=100)
6
7 n_epochs = 10
8 for epoch in range(n_epochs):
9     model.train()
10    for X_batch, y_batch in loader:
11        y_pred = model(X_batch)
12        loss = loss_fn(y_pred, y_batch)
13        optimizer.zero_grad()
14        loss.backward()
15        optimizer.step()
16    # Validation
17    model.eval()
18    y_pred = model(X_test)
19    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
```

```
20 print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

The MNIST dataset is small. This example should complete in a minute, with the output below. This simple network can produce accuracy at 92%.

```
1 Epoch 0: model accuracy 84.11%
2 Epoch 1: model accuracy 87.53%
3 Epoch 2: model accuracy 89.01%
4 Epoch 3: model accuracy 89.76%
5 Epoch 4: model accuracy 90.29%
6 Epoch 5: model accuracy 90.69%
7 Epoch 6: model accuracy 91.10%
8 Epoch 7: model accuracy 91.48%
9 Epoch 8: model accuracy 91.74%
10 Epoch 9: model accuracy 91.96%
```

Below is the complete code for the above multilayer perceptron classification on MNIST dataset.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision
5
6 # Load MNIST data
7 train = torchvision.datasets.MNIST('data', train=True, download=True)
8 test = torchvision.datasets.MNIST('data', train=True, download=True)
9
10 # each sample becomes a vector of values 0-1
11 X_train = train.data.reshape(-1, 784).float() / 255.0
12 y_train = train.targets
13 X_test = test.data.reshape(-1, 784).float() / 255.0
14 y_test = test.targets
15
16 class Baseline(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.layer1 = nn.Linear(784, 784)
20         self.act1 = nn.ReLU()
21         self.layer2 = nn.Linear(784, 10)
22
23     def forward(self, x):
24         x = self.act1(self.layer1(x))
25         x = self.layer2(x)
26         return x
27
28 model = Baseline()
29
30 optimizer = optim.SGD(model.parameters(), lr=0.01)
31 loss_fn = nn.CrossEntropyLoss()
32 loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=100)
33
34 n_epochs = 10
35 for epoch in range(n_epochs):
36     model.train()
37     for X_batch, y_batch in loader:
38         y_pred = model(X_batch)
39         loss = loss_fn(y_pred, y_batch)
40         optimizer.zero_grad()
41         loss.backward()
42         optimizer.step()
43     # Validation
44     model.eval()
45     y_pred = model(X_test)
46     acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
47     print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Simple Convolutional Neural Network for MNIST

Now that you have seen how to use multilayer perceptron model to classify MNIST dataset. Let's move on to try a convolutional neural network model. In this section, you will create a simple CNN for MNIST that demonstrates how to use all the aspects of a modern CNN implementation, including convolutional layers, pooling layers, and dropout layers.

In PyTorch, convolutional layers are supposed to work on images. Tensors for images should be the pixel values with the dimensions (sample, channel, height, width) but when you load images using libraries such as PIL, the pixels are usually presented as array of dimensions (height, width, channel). The conversion to a proper tensor format can be done using a transform from the torchvision library.

```
1 ...
2 transform = torchvision.transforms.Compose([
3     torchvision.transforms.ToTensor(),
4     torchvision.transforms.Normalize((0,), (128,)),
5 ])
6 train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
7 test = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
8 trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
9 testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)
```

You need to use DataLoader because the transform is applied when you read the data from the DataLoader.

Next, define your neural network model. Convolutional neural networks are more complex than standard multilayer perceptrons, so you will start by using a simple structure that uses all the elements for state-of-the-art results. Below summarizes the network architecture.

1. The first hidden layer is a convolutional layer, `nn.Conv2d()`. The layer turns a grayscale image into 10 feature maps, with the filter size of 5×5 and a ReLU activation function. This is the input layer that expects images with the structure outlined above.
2. Next is a pooling layer that takes the max, `nn.MaxPool2d()`. It is configured with a pool size of 2×2 with stride 1. What it does is to take the maximum in a 2×2 pixel patch per channel and assign the value to the output pixel. The result is a 27×27-pixels feature map per channel.
3. The next layer is a regularization layer using dropout, `nn.Dropout()`. It is configured to randomly exclude 20% of neurons in the layer in order to reduce overfitting.
4. Next is a layer that converts the 2D matrix data to a vector, using `nn.Flatten`. There are 10 channels from its input and each channel's feature map has size 27×27. This layer allows the output to be processed by standard, fully connected layers.
5. Next is a fully connected layer with 128 neurons. ReLU activation function is used.
6. Finally, the output layer has ten neurons for the ten classes. You can transform the output into probability-like predictions by applying a softmax function on it.

This model is trained using cross entropy loss and the Adam optimization algorithm. It is implemented as follows:

```
1 class CNN(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
5         self.relu1 = nn.ReLU()
6         self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
7         self.dropout = nn.Dropout(0.2)
8         self.flat = nn.Flatten()
9         self.fc = nn.Linear(27*27*10, 128)
10        self.relu2 = nn.ReLU()
11        self.output = nn.Linear(128, 10)
12
13    def forward(self, x):
14        x = self.relu1(self.conv(x))
15        x = self.pool(x)
16        x = self.dropout(x)
17        x = self.relu2(self.fc(self.flat(x)))
18        x = self.output(x)
19        return x
20
21 model = CNN()
22
23 optimizer = optim.Adam(model.parameters(), lr=0.01)
24 loss_fn = nn.CrossEntropyLoss()
25
26 n_epochs = 10
27 for epoch in range(n_epochs):
28     model.train()
29     for X_batch, y_batch in trainloader:
30         y_pred = model(X_batch)
31         loss = loss_fn(y_pred, y_batch)
32         optimizer.zero_grad()
33         loss.backward()
34         optimizer.step()
35     # Validation
36     model.eval()
37     acc = 0
38     count = 0
39     for X_batch, y_batch in testloader:
40         y_pred = model(X_batch)
41         acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
42         count += len(y_batch)
43     acc = acc / count
44     print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Running the above takes a few minutes and produces the following:

```
1 Epoch 0: model accuracy 81.74%
2 Epoch 1: model accuracy 85.38%
3 Epoch 2: model accuracy 86.37%
4 Epoch 3: model accuracy 87.75%
5 Epoch 4: model accuracy 88.00%
6 Epoch 5: model accuracy 88.17%
7 Epoch 6: model accuracy 88.81%
8 Epoch 7: model accuracy 88.34%
9 Epoch 8: model accuracy 88.86%
10 Epoch 9: model accuracy 88.75%
```

Not the best result but this demonstrates how convolutional layer works.

Below is the complete code for using the simple convolutional network.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision
5
6 # Load MNIST data
7 transform = torchvision.transforms.Compose([
```

```

8     torchvision.transforms.ToTensor(),
9     torchvision.transforms.Normalize((0,), (128,)),
10 ])
11 train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
12 test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
13 trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
14 testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)
15
16 class CNN(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
20         self.relu1 = nn.ReLU()
21         self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
22         self.dropout = nn.Dropout(0.2)
23         self.flat = nn.Flatten()
24         self.fc = nn.Linear(27*27*10, 128)
25         self.relu2 = nn.ReLU()
26         self.output = nn.Linear(128, 10)
27
28     def forward(self, x):
29         x = self.relu1(self.conv(x))
30         x = self.pool(x)
31         x = self.dropout(x)
32         x = self.relu2(self.fc(self.flat(x)))
33         x = self.output(x)
34         return x
35
36 model = CNN()
37
38 optimizer = optim.Adam(model.parameters())
39 loss_fn = nn.CrossEntropyLoss()
40
41 n_epochs = 10
42 for epoch in range(n_epochs):
43     model.train()
44     for X_batch, y_batch in trainloader:
45         y_pred = model(X_batch)
46         loss = loss_fn(y_pred, y_batch)
47         optimizer.zero_grad()
48         loss.backward()
49         optimizer.step()
50     # Validation
51     model.eval()
52     acc = 0
53     count = 0
54     for X_batch, y_batch in testloader:
55         y_pred = model(X_batch)
56         acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
57         count += len(y_batch)
58     acc = acc / count
59     print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

LeNet5 for MNIST

The previous model has only one convolutional layer. Of course, you can add more to make a deeper model. One of the earliest demonstration of the effectiveness of convolutional layers in neural networks is the “LeNet5” model. This model is developed to solve the MNIST classification problem. It has three convolutional layers and two fully connected layer to make up five trainable layers in the model, as it is named.

At the time it was developed, using hyperbolic tangent function as activation is common. Hence it is used here. This model is implemented as follows:

```

1 class LeNet5(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
5         self.act1 = nn.Tanh()
6         self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
7
8         self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
9         self.act2 = nn.Tanh()
10        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
11
12        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
13        self.act3 = nn.Tanh()
14
15        self.flat = nn.Flatten()
16        self.fc1 = nn.Linear(1*1*120, 84)
17        self.act4 = nn.Tanh()
18        self.fc2 = nn.Linear(84, 10)
19
20    def forward(self, x):
21        # input 1x28x28, output 6x28x28
22        x = self.act1(self.conv1(x))
23        # input 6x28x28, output 6x14x14
24        x = self.pool1(x)
25        # input 6x14x14, output 16x10x10
26        x = self.act2(self.conv2(x))
27        # input 16x10x10, output 16x5x5
28        x = self.pool2(x)
29        # input 16x5x5, output 120x1x1
30        x = self.act3(self.conv3(x))
31        # input 120x1x1, output 84

```



```

32     x = self.act4(self.fc1(self.flat(x)))
33     # input 84, output 10
34     x = self.fc2(x)
35     return x

```

Compare to the previous model, LeNet5 does not have Dropout layer (because Dropout layer was invented several years after LeNet5) and use average pooling instead of max pooling (i.e., for a patch of 2×2 pixels, it is taking average of the pixel values instead of taking the maximum). But the most notable characteristic of LeNet5 model is that it uses strides and paddings to reduce the image size from 28×28 pixel down to 1×1 pixel while increasing the number of channels from a one (grayscale) into 120.

Padding means to add pixels of value 0 at the border of the image to make it a bit larger. Without padding, the output of a convolutional layer will be smaller than its input. The stride parameter controls how much the filter should move to produce the next pixel in the output. Usually it is 1 to preserve the same size. If it is larger than 1, the output is a **downsampling** of the input. Hence you see in the LeNet5 model, stride 2 was used in the pooling layers to make, for example, a 28×28-pixel image into 14×14.

Training this model is same as training the previous convolutional network model, as follows:

```

1  ...
2  model = LeNet5()
3
4  optimizer = optim.Adam(model.parameters())
5  loss_fn = nn.CrossEntropyLoss()
6
7  n_epochs = 10
8  for epoch in range(n_epochs):
9      model.train()
10     for X_batch, y_batch in trainloader:
11         y_pred = model(X_batch)
12         loss = loss_fn(y_pred, y_batch)
13         optimizer.zero_grad()
14         loss.backward()
15         optimizer.step()
16     # Validation
17     model.eval()
18     acc = 0
19     count = 0
20     for X_batch, y_batch in testloader:
21         y_pred = model(X_batch)
22         acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
23         count += len(y_batch)
24     acc = acc / count
25     print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Running this you may see:

```

1  Epoch 0: model accuracy 89.46%
2  Epoch 1: model accuracy 93.14%
3  Epoch 2: model accuracy 94.69%
4  Epoch 3: model accuracy 95.84%
5  Epoch 4: model accuracy 96.43%
6  Epoch 5: model accuracy 96.99%
7  Epoch 6: model accuracy 97.14%
8  Epoch 7: model accuracy 97.66%
9  Epoch 8: model accuracy 98.05%
10 Epoch 9: model accuracy 98.22%

```

Here, we achieved accuracy beyond 98%.

The following is the complete code.

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  import torchvision
5
6  # Load MNIST data
7  transform = torchvision.transforms.Compose([
8      torchvision.transforms.ToTensor(),
9      torchvision.transforms.Normalize((0,),(128,)),
10 ])
11 train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
12 test = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
13 trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
14 testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)
15
16 class LeNet5(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
20         self.act1 = nn.Tanh()
21         self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
22
23         self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
24         self.act2 = nn.Tanh()
25         self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
26
27         self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
28         self.act3 = nn.Tanh()
29

```

```

30     self.flat = nn.Flatten()
31     self.fc1 = nn.Linear(1*1*120, 84)
32     self.act4 = nn.Tanh()
33     self.fc2 = nn.Linear(84, 10)
34
35     def forward(self, x):
36         # input 1x28x28, output 6x28x28
37         x = self.act1(self.conv1(x))
38         # input 6x28x28, output 6x14x14
39         x = self.pool1(x)
40         # input 6x14x14, output 16x10x10
41         x = self.act2(self.conv2(x))
42         # input 16x10x10, output 16x5x5
43         x = self.pool2(x)
44         # input 16x5x5, output 120x1x1
45         x = self.act3(self.conv3(x))
46         # input 120x1x1, output 84
47         x = self.act4(self.fc1(self.flat(x)))
48         # input 84, output 10
49         x = self.fc2(x)
50         return x
51
52 model = LeNet5()
53
54 optimizer = optim.Adam(model.parameters())
55 loss_fn = nn.CrossEntropyLoss()
56
57 n_epochs = 10
58 for epoch in range(n_epochs):
59     model.train()
60     for X_batch, y_batch in trainloader:
61         y_pred = model(X_batch)
62         loss = loss_fn(y_pred, y_batch)
63         optimizer.zero_grad()
64         loss.backward()
65         optimizer.step()
66     # Validation
67     model.eval()
68     acc = 0
69     count = 0
70     for X_batch, y_batch in testloader:
71         y_pred = model(X_batch)
72         acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
73         count += len(y_batch)
74     acc = acc / count
75     print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Resources on MNIST

The MNIST dataset is very well studied. Below are some additional resources you might want to look into.

- Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. [The MNIST database of handwritten digits](#).
- Rodrigo Benenson. [What is the class of this image? Classification datasets results](#), 2016.
- Digit Recognizer: Learn computer vision fundamentals with the famous MNIST data. [Kaggle](#).
- Hubert Eichner. [Neural Net for Handwritten Digit Recognition in JavaScript](#).

Summary

In this post, you discovered the MNIST handwritten digit recognition problem and deep learning models developed in Python using the Keras library that are capable of achieving excellent results. Working through this chapter, you learned:

- How to load the MNIST dataset in PyTorch with torchvision
- How to convert the MNIST dataset into PyTorch tensors for consumption by a convolutional neural network
- How to use PyTorch to create convolutional neural network models for MNIST
- How to implement the LeNet5 model for MNIST classification

Get Started on Deep Learning with PyTorch!

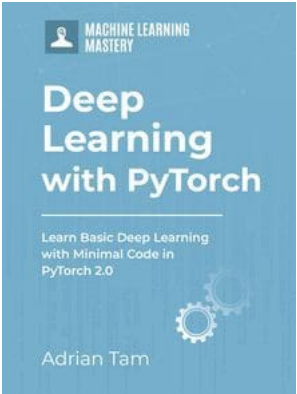
Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
[Deep Learning with PyTorch](#)

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises



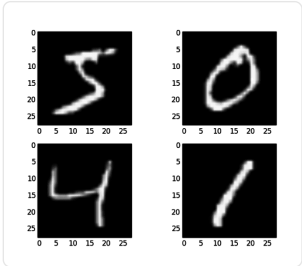
SEE WHAT'S INSIDE

Share

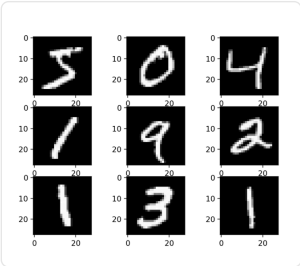
Share

Post

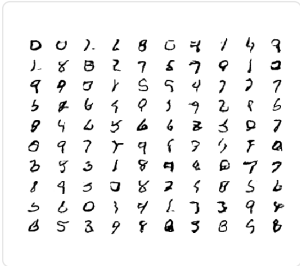
More On This Topic



Handwritten Digit Recognition Using Convolutional...



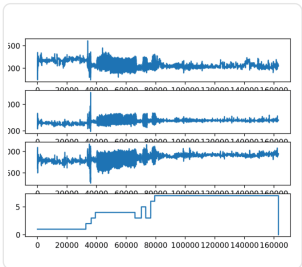
How to Develop a CNN for MNIST Handwritten Digit...



How to Develop a GAN for Generating MNIST Handwritten Digits



How to Do Named Entity Recognition (NER) with a BERT Model



A Gentle Introduction to a Standard Human Activity...



Evaluate Machine Learning Algorithms for Human...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

< Building a Convolutional Neural Network in PyTorch

LSTM for Time Series Prediction in PyTorch >

4 Responses to *Handwritten Digit Recognition with LeNet5 Model in PyTorch*

**Patrick Wood** May 31, 2023 at 1:45 pm #

REPLY ↩

I think train should be False for the test dataset.

**Eduardo Chico** November 12, 2023 at 7:18 am #

REPLY ↩

haha came here exactly to make the same observation

**Michael Hogg** December 20, 2024 at 8:54 am #

REPLY ↩

Thanks for the article.

One question for the data transform:

```
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,),(128,)),
])
```

The torchvision docs say that ToTensor transforms the image data from the range [0,255] to [0.0,1.0]. This is fine, but then applying Normalize((0,),(128,)) does not make sense, since this means divide by 128.

I've seen other recommendations of:

```
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0.5,),(0.5,)),
])
```

which will scale the image data to [-1,1].

**James Carmichael** December 20, 2024 at 9:54 am #

REPLY ↩

Hi Michael...You're absolutely correct in your observation, and the use of Normalize((0,),(128,)) in the provided transformation pipeline does not align with the usual intent of data normalization.

Here's a detailed explanation:

What ToTensor() Does

– The ToTensor() transformation converts image data from a PIL Image or NumPy array format in the range [0, 255] to a PyTorch tensor in the range [0.0, 1.0].

What Normalize(mean, std) Does

– Normalize((mean,),(std,)) applies the transformation:

$$\text{output} = \frac{\text{input} - \text{mean}}{\text{std}}$$

Where:

- mean is the mean value to subtract.
- std is the standard deviation value by which to divide.

Why Normalize((0,),(128,)) Doesn't Make Sense

Since the data has already been scaled to [0.0, 1.0] by ToTensor(), applying Normalize((0,),(128,)) means dividing the scaled values by 128. This is likely a mistake or misunderstanding of the normalization process. Instead, you typically normalize the data to have a mean of 0.0 and a standard deviation of 1.0 by using appropriate values for mean and std.

The Recommended Normalize((0.5,),(0.5,))

- Using Normalize((0.5,),(0.5,)) scales the data from [0.0, 1.0] to [-1.0, 1.0].
- First, subtracting 0.5 shifts the data to [-0.5, 0.5].
- Then, dividing by 0.5 scales it to [-1.0, 1.0].

This is a common practice for models that work well with input data in the range [-1, 1].

Correct Transformation for LeNet-5

If you're following a standard LeNet-5 implementation, it's better to use:

```
python
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0.5,),(0.5,)), # Scales data to [-1, 1]
])
```

Alternatively, if you're aiming for zero mean and unit variance normalization based on a dataset's statistics (e.g., MNIST), you can use:

python

```
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0.1307,), (0.3081,)), # MNIST's mean and std
])
```

These statistics (mean=0.1307, std=0.3081) are computed from the MNIST dataset itself and are widely used in practice.

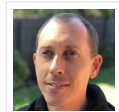
Summary

- `**Normalize((0,), (128,))` is incorrect unless there's a specific reason to divide by 128 (unlikely here).
- Use `**Normalize((0.5,), (0.5,))` for scaling to [-1, 1], or compute dataset-specific mean and std if zero-mean, unit-variance normalization is required.
- Choose the normalization based on the input expectations of the LeNet-5 implementation you're using.

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The **Deep Learning with PyTorch** EBook
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved